

Kubernetes Horizontal Pod Autoscaler (HPA):

1. HPA Architecture and Core Concepts

The **Kubernetes Horizontal Pod Autoscaler (HPA)** is a built-in controller that automatically adjusts the number of pod replicas in a workload (like a Deployment or StatefulSet) based on observed metrics such as CPU, memory, or custom/external signals.

The HPA's core function is to automatically adjust (scale) the number of Pod replicas for a target workload—such as a Deployment or StatefulSet—based on observed resource utilization and application metrics.

In essence, the HPA provides the mechanism for **horizontal scaling**, meaning the system responds to increasing demand by deploying *more Pods*, rather than simply giving more resources to existing Pods (vertical scaling).

The HPA's primary goal is to continuously monitor metrics (like CPU, memory, or custom signals) and calculate the necessary replica count to maintain a predefined target utilization level, thereby mapping the dynamic reality of application demand to the declarative state management of Kubernetes.

1.1. Horizontal vs. Vertical Scaling:

Vertical scaling (often managed by the Vertical Pod Autoscaler, VPA) involves assigning greater resources (CPU and memory) to the Pods already running for a workload.

In contrast, horizontal scaling—the domain of the HPA—responds to increased load by deploying additional Pods.

For distributed, stateless microservices, horizontal scaling is architecturally preferred as it enhances resilience, improves load distribution, and maximizes resource utilization.

1.2. HPA Resource Model and Prerequisites

Modern autoscaling configurations should leverage the autoscaling/v2 API version, which provides extensive support for multiple **metric types (Resource, Custom, External)** and the essential advanced behavior tuning configurations. The operation of the HPA is inherently dependent on accurate, accessible metric data.

A foundational prerequisite for HPA function, specifically for resource-based scaling (CPU and Memory), is the **Metrics Server**. This component collects resource utilization metrics from the Kubelets across the cluster and exposes them via the metrics.k8s.io API. HPA queries this API every **sync period (default 15 seconds)**. Without this server deployed and configured, native CPU and memory utilization metrics are unavailable, rendering basic HPA scaling impossible. The dependency on the Metrics Server and the subsequent necessary integration of metrics pipelines for Custom or External metrics implies that scaling responsiveness is inherently limited not just by the HPA controller's cycle time, but by the slowest component in the metric collection chain, including the scrape interval and aggregation time.

Default Sync Period

By default, the HPA syncs and checks the current metric values every 15 seconds. This interval, configured globally via the `--horizontal-pod-autoscaler-sync-period` (on the controller manager), establishes the absolute minimum reaction time for the autoscaler, as the controller only wakes up and queries metrics at this frequency.

In managed Kubernetes (EKS, AKS, GKE), this setting is **not changeable**.

→ Even if your stabilization window is smaller (say 5s), new metrics still arrive only every **15s**, meaning scaling decisions rely **on the latest available metrics**, not necessarily fresh ones.

2. The HPA Control Loop and Scaling Algorithm

The HPA operates as a controller within the kube-controller-manager, continuously executing a control loop to monitor metrics and adjust replica counts.

2.1. The Operational Cycle: Polling, Calculation, and Action

The operational cycle of the HPA controller begins with metric polling. By default, the HPA syncs and checks the current metric values every **15 seconds**.

When the observed metric values exceed the target threshold defined in the HPA specification, the HPA initiates a sequence:

→ first, it computes the desired replica count, then it applies any configured scaling policies, and finally, it updates the target workload resource (Deployment or StatefulSet) with the new replica count.

2.2. The Desired Replicas Formula:

The central logic of the HPA controller is a ratio-based algorithm applied independently for each configured metric. This calculation determines the desired replicas to distribute the load

within the pods to maintain the target metrics.

$\text{desiredReplicas} = \text{ceil}[\text{currentReplicas} \times (\text{currentMetricValue} / \text{targetMetricValue})]$

For instance, if the target CPU utilization is 60% and the current average utilization is 90%, the ratio is $90 / 60 = 1.5$. This indicates the workload is running at 1.5 times the target capacity, and therefore requires a 1.5 multiplier on the current replica count.

A critical feature of this calculation is the use of the **ceiling function**, which always rounds the result up to the nearest whole number.

Scale-Up Example

Consider a scenario where the workload is under pressure:

If a workload has 5 running replicas, a target CPU of 60%, and an observed CPU of 90%:

- Ratio = $90 / 60 = 1.5$
- Desired Replicas = $\text{Ceiling}(5 \times 1.5) = \text{Ceiling}(7.5) = 8$ replicas.

Scale-Down Example

Conversely, if the current resource usage drops significantly below the target, the HPA determines a scale-down:

If the current resource usage is 50m and the desired target value is 100m, the ratio is $50 / 100 = 0.5$. This ratio results in halving the number of replicas

2.3. The Role of Metric Tolerance

To prevent “flapping” due to small metric changes, Kubernetes introduces a **tolerance threshold**.

The default value for this configurable tolerance is **0.1 (10%)**.

Range: $[0.9 \times \text{target}, 1.1 \times \text{target}]$

Scaling happens **only** when metrics go **outside** this band i.e the control plane explicitly skips any scaling action if the calculated ratio of observed metric to target metric is sufficiently close to 1.0.

—> Even if **desiredReplicas** suggests scaling up (based on math), the **tolerance filter** takes precedence.

Scaling **will not happen** unless metric deviation $> 10\%$.

- **Tolerance has higher priority than desired state.**

This tolerance parameter acts as a critical damping factor, stabilizing the system. Without it, noisy metrics close to the target threshold would constantly trigger the HPA control loop, placing undue, high-frequency write load on the Kubernetes API server, potentially destabilizing the entire control plane. The tolerance configuration is thus a fundamental defense mechanism against control plane instability driven by application metrics.

Regardless of how intense the demand is or how low the observed metrics drop, the replica count calculated by the HPA will **never** exceed the `maxReplicas` limit, and it will **never** fall below the `minReplicas` limit.

3. Metric Sources and APIs in HPA v2

The HPA v2 API provides extensibility by allowing scaling based on four distinct metric types, each sourced from a specific Kubernetes API endpoint. The choice of metric type dictates the configuration complexity and the necessary infrastructure dependencies.

3.1. Resource Metrics (type: Resource)

Resource metrics are the most straightforward type, focused on CPU and Memory utilization. These metrics relate directly to the resource requests and limits defined on the Pods. The data source for these metrics is the Metrics Server, which exposes data via the `metrics.k8s.io` API. Scaling targets are typically defined using `AverageUtilization` (a percentage of the resource requested by the container) or `AverageValue` (the raw measured value in millicores or bytes).

3.2. Custom Metrics (type: Pods or type: Object)

Custom metrics enable scaling based on application-specific data, such as HTTP requests per second or queue length. These metrics must be intrinsically tied to a specific Kubernetes object (a Pod, Deployment, or Service).⁸ The data is accessed via the Custom Metrics API (`custom.metrics.k8s.io`), which requires an intermediary adapter, such as the Prometheus Adapter, to expose application-level metrics to the Kubernetes control plane.⁸

When using `type: Pods`, the HPA scales the target to maintain a uniform average value across all running Pods. The configuration requires specifying the custom metric name and the target average value.

3.3. External Metrics (type: External)

External metrics offer the highest degree of flexibility by allowing scaling based on signals

that originate entirely outside the Kubernetes cluster and are not tied to any internal Kubernetes object. Examples include cloud service queue depths (e.g., AWS SQS or Kafka lag) or global database connection counts.

External metrics are sourced via the External Metrics API (external.metrics.k8s.io) and also require specialized adapters. A key distinction is that these metrics do not require specific Kubernetes object labels (like pod or namespace) for targeting.⁴ This decoupling of scalability from the workload's immediate resource status is highly advantageous as it allows for **proactive scaling**. For example, scaling based on an increasing upstream queue size allows new replicas to be provisioned *before* the application's internal CPU or RPS metrics show strain, significantly improving responsiveness during traffic spikes.

3.4. Container Resource Metrics

The HPA API also supports tracking resource usage of individual containers within a multi-container Pod, rather than relying solely on the average resource usage of the entire Pod. This feature is essential for workloads where a scaling decision should be driven only by the primary application container, while ignoring the potentially heavy but transient resource usage of sidecar containers (e.g., log aggregators or service meshes).

The requirement to implement external adapters and configure custom rules for Custom and External metrics introduces significant administrative overhead and increases the number of potential failure points compared to simple Resource metrics. However, this complexity is an accepted trade-off for gaining fine-grained control over scaling criteria.

Table 4: HPA Metric Types and Corresponding APIs

Metric Type	HPA Spec Type	Description	API Service Used
Resource	Resource	CPU or Memory utilization tied directly to resource requests/limits.	metrics.k8s.io (Metrics Server)
Custom (Per Pod)	Pods	Metrics associated with individual Pods, averaged across the target workload.	custom.metrics.k8s.io
Custom (Per Object)	Object	Metrics associated with the workload	custom.metrics.k8s.io

		resource itself.	
External	External	Metrics originating outside the Kubernetes control plane, not tied to any internal object.	external.metrics.k8s.io

4. Scaling Behavior (.spec.behavior)

HPA v2 introduced **behavior tuning**, which governs *how fast* and *how frequently* HPA scales, and prevent the problematic phenomenon known as "flapping" —the rapid oscillation of replicas scaling up and immediately scaling down using:

- **stabilizationWindowSeconds**
- **policies** (Pods or Percent)
- **selectPolicy**

Crucially, the rules defined within the behavior field are applied *after* the primary scaling algorithm has calculated the Desired Replicas.

4.1. Stabilization Window (stabilizationWindowSeconds)

The stabilization window defines how long HPA should **look back** before making a scaling decision as to prevent transient metric spikes or dips from triggering immediate, unnecessary scaling actions. The HPA controller maintains a history of past desired replica counts calculated within this window.

- **Scale-Down Stabilization:** For scaling down, a long window is critical. A standard production configuration uses **300 seconds (5 minutes)**. This dictates that the HPA will select the *highest desired replica* count observed within the previous five minutes, effectively delaying any scale-down request until the load reduction is confirmed to be sustained.
- **Scale-Up Stabilization:** While some aggressive configurations may omit scale-up stabilization, a short window (e.g., 60 seconds) can prevent "panic scaling". Will select the **lowest desired** replica count from the last window to avoid rapid upscaling.

Edge Case 1:

If scale-up stabilization window = 5s but metric update = every 15s

→ HPA still uses the **most recent metric**

You might get delayed scaling because **fresh data isn't yet available**.

Always set your **stabilization window** $\geq$ **metrics collection interval**, especially for **scale-up** actions that are time-sensitive.

Edge Case 2:

If during the 30s stabilization period, metrics show pods = [2, 3, 4],

HPA picks **min(2,3,4)** for scale-up (most conservative).

For scale-down, it picks **max(2,3,4)** (most stable).

4.2. Scaling Policies (policies):

HPA scaling **policies** are part of the **behavior** field.

They let you **control how fast** scaling happens — i.e., how many pods can be added or removed in a given time window.

Without policies, HPA may scale aggressively (many pods up/down instantly).

Policies let you **rate-limit** those changes for smoother scaling. These policies are enforced over a specific time window defined by **periodSeconds** (with a maximum configurable value of 1800 seconds, or half an hour).

Policy Type	Meaning	Constraint Example
Pods	Limits the scaling change to a fixed number of pods.	Allows at most 4 replicas to be scaled down in 60 seconds.
Percent	Limits the scaling change to a percentage of current replicas.	Allows at most 10% of the current replicas to be scaled down in 60 seconds.

- type: **Pods**

value: 4

periodSeconds: 60

→ HPA can add/remove **at most 4 pods** in 60 seconds.

- type: **Percent**

value: 10

periodSeconds: 60

→ HPA can change **up to 10%** of current replicas every 60 seconds.

4.3. Overriding the Selection (**selectPolicy**)

When multiple policies are defined (e.g., both **Pods** and **Percent**), the HPA must choose which limit to apply during its synchronization cycle.

By default behavior, the HPA selects the policy that allows the **highest amount of change** (the largest number of pods to be added or removed).

The default preference for the maximum change can be overridden using the **selectPolicy** field for either the **scaleUp** or **scaleDown** direction:

- **selectPolicy: Min:** Selects the policy that allows the **smallest change** in the replica count. This is typically used for **scaleDown** to ensure stability and gradual resource shedding.
- **selectPolicy: Disabled:** This option provides a mechanism to temporarily and selectively prevent scaling in a specific direction (up or down) without altering the underlying metric targets or stabilization windows.

If you have 80 replicas:

- 10% of 80 = 8 pods per minute
- Fixed 4 pods per minute
→ The **Percent policy** allows more change → HPA will pick **8 pods per minute**.

Table : HPA Scaling Policy Configuration Elements

Field	Purpose	Typical Values / Behavior	Control Direction
stabilizationWindowSeconds	Time period to look back at past replica counts, preventing erratic changes.	60s (Scale-Up), 300s (Scale-Down)	Velocity Damping
periodSeconds	The time interval (in seconds) over which a	15 or 60 seconds	Velocity Limiting

	policy limit applies.		
type: Pods	Limits scaling by an absolute number of pods.	value: Integer count ¹¹	Scale Velocity
type: Percent	Limits scaling by a percentage of current running replicas.	value: Integer percentage ¹¹	Scale Velocity
selectPolicy	Determines which rule to enforce if multiple policies are defined.	Min, Max, or Disabled ¹¹	Conflict Resolution

5. Examples

Case 1: Conflicting Metrics (CPU vs Memory)

If HPA has multiple metrics (e.g., CPU and Memory):

```
metrics:
- type: Resource
  resource:
    name: cpu
    target:
      type: Utilization
      averageUtilization: 70
- type: Resource
  resource:
    name: memory
    target:
      type: Utilization
      averageUtilization: 80
```

CPU = 90% (target 70%) → suggests scale-up

Memory = 50% (target 80%) → no scale

HPA scales up because the highest need wins.

- Each metric produces its own desiredReplica count.
- Final replicas = maximum of all calculated values.
This ensures that if *any resource* is under pressure, HPA scales up.

Case 2: Tolerance deciding scaling

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: myapp-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: myapp
  minReplicas: 2
  maxReplicas: 10
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 50
```

target CPU = 50%

With 10% tolerance →

- Lower bound = $50 \times (1 - 0.1) = 45\%$
- Upper bound = $50 \times (1 + 0.1) = 55\%$

So, as long as CPU utilization is **between 45% and 55%**, the HPA will **not** scale up or down.

Case 3: Behavior Tuning

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: tuned-web-app-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: web-app
  minReplicas: 3
  maxReplicas: 25
  metrics:
  - type: Resource
    resource:
      name: cpu
```

```
target:
  type: Utilization
  averageUtilization: 70
behavior:
  scaleUp:
    stabilizationWindowSeconds: 60
  policies:
    - type: Pods
      value: 4
      periodSeconds: 60
    - type: Percent
      value: 100
      periodSeconds: 60
    selectPolicy: Max
  scaleDown:
    stabilizationWindowSeconds: 300
  policies:
    - type: Pods
      value: 1
      periodSeconds: 15
    - type: Percent
      value: 50
      periodSeconds: 15
    selectPolicy: Min
```

The scale-down configuration illustrates the powerful interaction between stabilization and policy limits. The `stabilizationWindowSeconds: 300` ensures that the decision to scale down is delayed until the low load is sustained for five minutes. Once the decision is made, the `selectPolicy: Min` enforces the safest rule, which in this case is `type: Pods, value: 1` every `periodSeconds: 15`. This enforces a controlled, serialized removal of replicas, maximizing stability during traffic decline

Case 4: MinMaxReplicas

```
minReplicas: 1
maxReplicas: 4
behavior:
  scaleUp:
    policies:
      - type: Pods
        value: 4
        periodSeconds: 60
      - type: Percent
        value: 10
        periodSeconds: 60
```

I have current replicas as 2

Apply Scaling Policies

Pods Policy: allows up to +4 pods in 60 seconds

→ From 2 pods, this would allow $2 + 4 = 6$ pods maximum.

Percent Policy: allows up to +10% of 2 pods = 0.2 pods (→ rounded down to 0 or 1)

→ So very small change — practically negligible.

- By default, **selectPolicy: Max** for scale-up.
So HPA picks the *larger* of the two allowed changes.

→ The Pods policy allows a bigger jump (up to +4 pods).

After calculating the desired replicas and applying the policies,

HPA must respect the global limits:

Final replicas = $\min(\max(\text{desiredReplicas}, \text{minReplicas}), \text{maxReplicas})$

Now: Desired replicas (from metrics) = 5

Policy allows up to 6

But **maxReplicas = 4**

→ So HPA limits scaling to **4 replicas**.

Even if a **policy** allows scaling higher,

HPA **never violates** the **maxReplicas** boundary.

The scaling policies only *control the rate of scaling* —

they **don't override** the replica boundaries set by min/max.

6. Conclusions

The Horizontal Pod Autoscaler is a highly effective tool for achieving elasticity in Kubernetes, but its expert operation requires careful attention to architectural constraints and advanced tuning parameters.

1. **Metrics Collection:** The HPA controller wakes up every **15 seconds**—the absolute minimum reaction time—to query the Metrics API for data (CPU, Custom, External). This sync period is the primary architectural constraint on responsiveness.
2. **Apply Tolerance:** The process first checks the default 10% buffer to determine if the metric variance is significant enough to warrant *any* action, preventing unnecessary scaling based on marginal metric fluctuations.
3. **Calculate Desired Replicas:** If action is needed, the HPA computes the ideal replica count using a ratio-based formula. This calculation incorporates a bias toward

availability by always using the **ceiling function** (rounding up) to slightly overshoot demand during scale-up.

4. **Apply Stabilization Window:** The HPA then consults a history of past desired replica counts. This asymmetric mechanism ensures scaling actions are based on sustained load trends, not transient spikes, preventing rapid oscillation ("flapping"). Scale-down, in particular, uses a long window (e.g., 300 seconds) to delay premature pod termination.
5. **Apply Scaling Policies:** Finally, velocity limits are enforced using defined policies (**Pods** or **Percent**). These rules restrict the *rate* at which replicas can be added or removed over a specified time window (**periodSeconds**). Conflict resolution occurs here, typically favoring the maximum change for scale-up and the minimum change (**selectPolicy: Min**) for stability during scale-down.

Mastery of the HPA hinges on configuring **asymmetric scaling behavior** in the **.spec.behavior** field: fast for scale-up (to meet demand) and slow for scale-down (to ensure stability). This deliberate operational asymmetry is the technical mechanism required to overcome the chaotic reality of production traffic.